

Projet Jarvis — Vers une architecture hybride Hermès

Claude Max + Hermes Agent + Ollama + Obsidian

Jarvis, assistant domotique de Mickael

24 avril 2026 — Session 36

Contents

Préambule	3
Chapitre 1 — Vue d'ensemble	4
1.1 Qu'est-ce que Jarvis aujourd'hui ?	4
1.2 Ce qui fonctionne bien	4
1.3 Où l'on veut aller	4
Chapitre 2 — Les briques technologiques	7
2.1 Hermes Agent (Nous Research, MIT)	7
2.2 Modèles locaux via Ollama (RTX 3090 24 Go)	8
2.3 Claude (via Plan Max, conservé)	9
2.4 OpenRouter (optionnel, pay-as-you-go)	9
2.5 Obsidian (mémoire partagée, optionnel)	9
Chapitre 3 — Comment ça fonctionne	11
3.1 Le principe du routing intelligent	11
3.2 Les fallbacks dynamiques	11
3.3 Les trois niveaux de mémoire	11
3.4 Les skills auto-crées	13
3.5 Les gateways — pilotage depuis n'importe où	13
Chapitre 4 — Ce que ça change pour toi	15
4.1 Répartition des appels LLM — avant / après	15
4.2 Cohabitation Cowork ↔ Hermès (pas de remplacement)	16
4.3 Pilotage mobile vraiment natif	16
4.4 Préservation du quota Max	16
Chapitre 5 — Coûts et retour sur investissement	17
5.1 Comparatif des stratégies d'abonnement	17
5.2 Ce qui a été écarté	18
5.3 Budget temps d'installation	18
5.4 Budget temps récurrent	18
Chapitre 6 — Phasage et timeline	19
6.1 Vue d'ensemble des phases	19
6.2 Détail de chaque phase	19
6.3 Points de décision côté Mickael	21
Chapitre 7 — Risques et garde-fous	22
7.1 La Règle 0 s'applique partout	22

7.2	Failure modes et mitigation	22
7.3	Plan de rollback	23
Conclusion		24
	Synthèse	24
	Prochaines actions	24
	Dernière précision	24
Annexes		26
	A — Glossaire technique	26
	B — Configuration PC Mickael	27
	C — Sources et références	27

Préambule

Ce document présente la vision d'ensemble du projet Jarvis à son état actuel (avril 2026) et la direction proposée pour les prochains mois. Il synthétise la réflexion conduite en session 36 autour du pattern "LLM Wiki" d'Andrej Karpathy, l'analyse du schéma "Hermès + Claude + Obsidian" envoyé par Mickael, et surtout la **découverte de Hermes Agent** (Nous Research, MIT, février 2026), produit open-source qui implémente nativement l'orchestration multi-LLM envisagée.

Il est organisé pour être lu **une fois de bout en bout** (lecture pédagogique) ou **consulté par chapitre** selon les décisions en cours.

La règle absolue de cadre reste la même : **aucune modification d'architecture Jarvis n'a été faite ni ne sera faite sans validation explicite de Mickael**. Ce document est une vision, pas une implémentation.

Chapitre 1 — Vue d'ensemble

1.1 Qu'est-ce que Jarvis aujourd'hui ?

Jarvis est l'assistant domotique personnel de Mickael, centré sur la gestion de sa maison connectée (Home Assistant à Seremange-Erzange) et la gestion de ses quatre boîtes email. Il fonctionne sur une architecture **100 % Claude** (Anthropic), animée par deux points d'entrée principaux :

- **Cowork Desktop** sur le PC Windows 11 Pro 24 h/24 — sessions riches avec accès fichiers, MCPs multiples, skills auto-déclenchées, mémoire persistante.
- **Claude Code CLI** (`claude -p headless`) — pour les tâches planifiées et les opérations stdio inaccessibles depuis Cowork.

Mickael interagit par **Cowork** quand il est au PC, et par **claude.ai mobile** quand il est en déplacement. Les tâches récurrentes (tri email, Mode Réactif, rapports journaliers) tournent via **Windows Task Scheduler** qui appelle Claude Code CLI.

L'infrastructure repose sur :

- **ha-mcp** (80+ outils Home Assistant via URL publique `mcp.might.ovh`)
- **Gmail MCP custom** (GongRzhe, stdio local, scope `gmail.modify`)
- **PDF Tools MCP** (`pdf-toolkit`, 21 outils, activé S35)
- **Claude in Chrome** (automation Brave pour Outlook et dashboards hors MCP)
- Arborescence locale de ~130-165 fichiers markdown (Ressources, skills, memory, auto-memory)

1.2 Ce qui fonctionne bien

Cette architecture est **fonctionnelle et stable**. En 6 semaines d'usage depuis la migration du 19 avril, elle a absorbé :

- L'installation et la mise en production du Mode Réactif (détection alertes HA → email → traitement CLI automatique)
- La refonte complète du tri email Gmail automatisé
- Le tri Outlook priorités
- Les refontes Home Assistant (aires, étages, dashboard)
- Un audit de sécurité complet (2 failles HAUTES fermées)

1.3 Où l'on veut aller

Le principal point de tension identifié est la **consommation du quota Claude Max**. Même si le plan est souscrit, chaque appel Claude consomme du quota, allonge le contexte chargé à

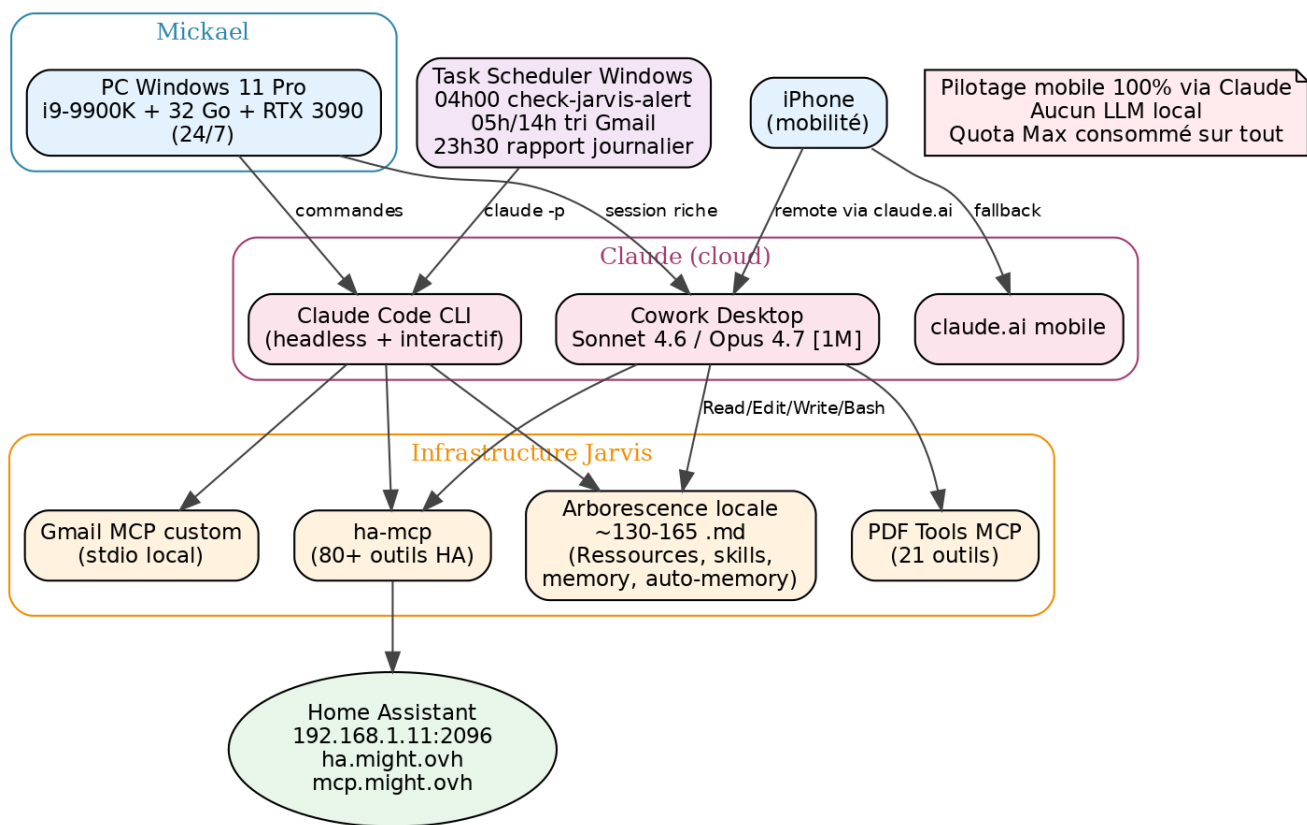


Figure 1: Architecture Jarvis actuelle — avril 2026

chaque session et introduit du “context rot” (dégradation des réponses sur les longs contextes). Par ailleurs, Jarvis est **dépendant du cloud** pour toute action — si Claude est down, Jarvis l’est aussi.

L’objectif du projet décrit ici est triple :

1. **Réduire significativement** la consommation de quota Max en déléguant les 70-80 % de tâches “standard” (tri, classification, résumé, brouillons) à des modèles locaux.
2. **Gagner en mobilité native** en exposant un point d’entrée dans l’app Home Assistant mobile (et Telegram) plutôt que via claude.ai fallback.
3. **Préparer une résilience locale** — si le cloud est indisponible, Jarvis continue à fonctionner en mode dégradé via les modèles locaux.

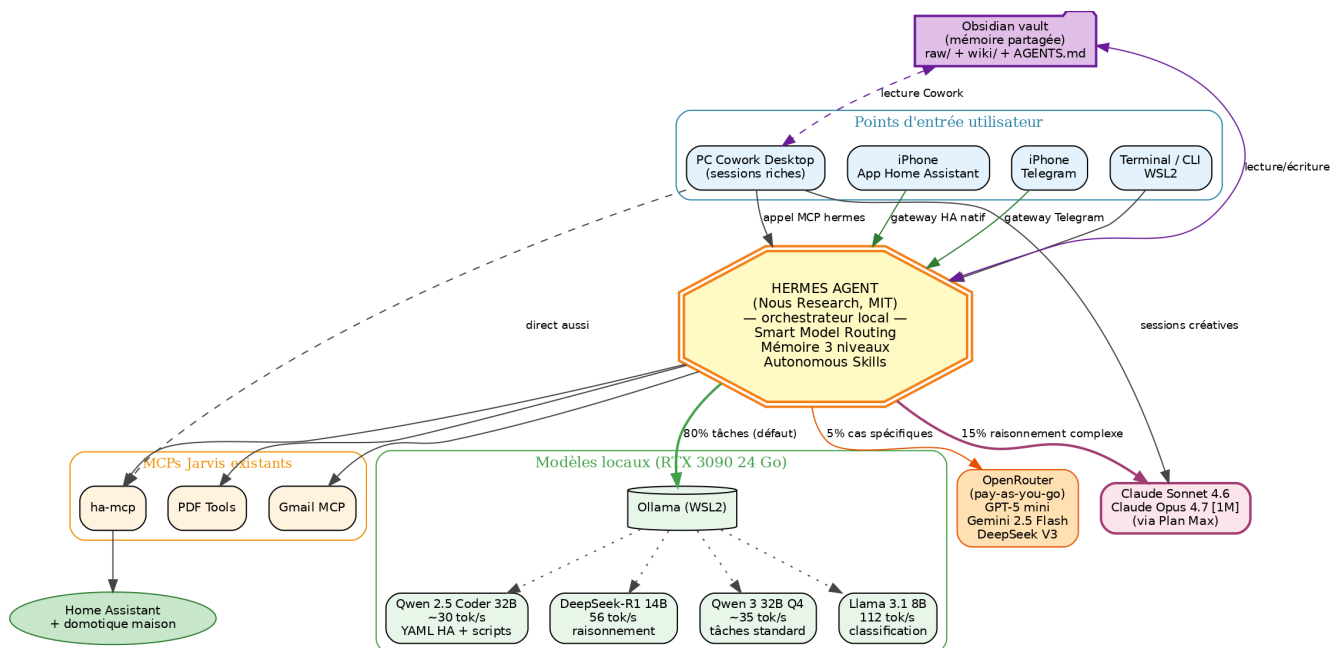


Figure 2: Architecture Jarvis cible — Claude + Hermès + Ollama + Obsidian

Chapitre 2 — Les briques technologiques

La solution repose sur **cinq briques** dont **aucune n'est à développer** — toutes existent, sont documentées, et ont une communauté d'utilisateurs active.

2.1 Hermes Agent (Nous Research, MIT)

Hermes Agent est un framework d'orchestrateur agentique lancé en **février 2026** par **Nous Research** (créateurs des modèles Llama fine-tunés Hermes). Licence **MIT**. Installation sur Linux, macOS, **WSL2** pour Windows, ou Android Termux.

Philosophie affichée : **“Harness Engineering”**. L'idée est que le levier technique de 2026 n'est plus un modèle plus intelligent, mais un meilleur **harnais** autour du modèle — cinq couches coordonnées : instructions, constraints, feedback, memory, orchestration.

Fonctionnalités natives clés :

- **Multi-LLM natif** : Anthropic (Claude), OpenAI (GPT), **OpenRouter (200+ modèles)**, Ollama local, vLLM, SGLang, Nous Portal, MiniMax, Kimi, tout endpoint OpenAI-compatible. Configuration dans `~/.hermes/config.yaml`.
- **MCP natif** : Hermès consomme les serveurs MCP existants (ha-mcp, gmail-mcp-local, pdf-toolkit) sans modification. Allowlist/denylist par outil via `tools.include` et `tools.exclude`.
- **Smart Model Routing (v0.10+)** : seuils complexité et valeur ajustables, règles de fallback vers Claude quand la requête dépasse les capacités du local. Monitoring ressources.
- **Mémoire à trois niveaux** avec recherche FTS5 full-text et résumés LLM :
 - *Session memory* — la conversation en cours
 - *Persistent memory* — faits et préférences entre sessions (équivalent `.auto-memory/`)
 - *Skill memory* — patterns de résolution réutilisables (équivalent `.claude/skills/` mais auto-générés)
- **Autonomous skill creation** — après une tâche complexe résolue, Hermès peut générer automatiquement une skill réutilisable.
- **Pattern LLM Wiki de Karpathy en backend** — Hermès utilise un git repo local pour maintenir un wiki markdown auto-mis à jour (Ingest / Query / Lint).
- **Gateways multi-canaux** — CLI, **Home Assistant natif**, **Telegram**, Discord, Slack, WhatsApp, Signal, Email, SMS, Matrix, iMessage (via BlueBubbles), webhooks, cron jobs.

AGENTS.md au root du vault joue le rôle du `CLAUDE.md` de Jarvis — schéma opérationnel partagé : identité, projets actifs, contraintes, préférences, historique.

Ressources officielles : [site](#), [doc](#), [repo GitHub](#).

2.2 Modèles locaux via Ollama (RTX 3090 24 Go)

Ollama est le runtime standard de facto pour l’inférence LLM locale. Installation simple (curl install sous WSL2), téléchargement de modèles via `ollama pull`, exposition via API HTTP locale consommable par Hermès.

Le GPU de Mickael — **NVIDIA RTX 3090 24 Go VRAM** — est dans la **sweet spot** 2026 pour l’inférence locale :

- Meilleur rapport qualité/prix (\$500-700 d’occasion en 2026)
- 24 Go VRAM = la même capacité qu’une RTX 4090, pour les modèles 32B quantifiés Q4
- Supporte les modèles 14B / 8B en pleine qualité avec marge de contexte

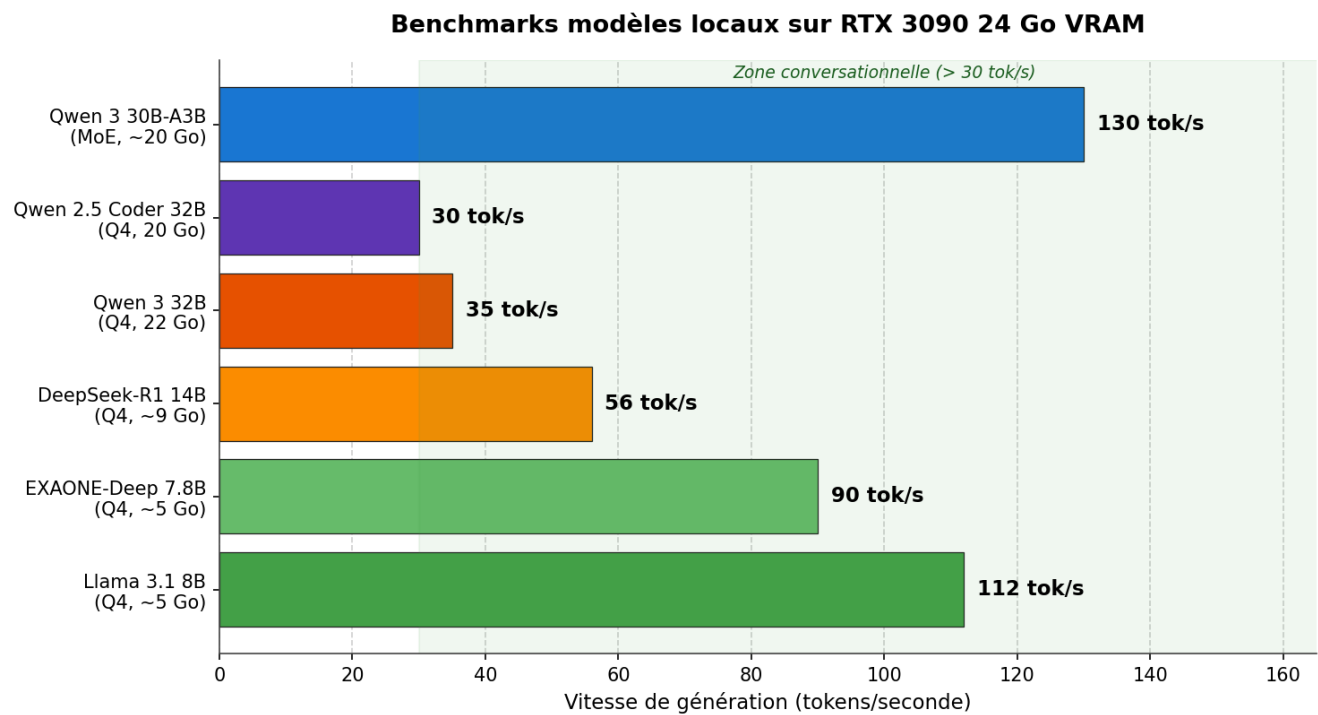


Figure 3: Benchmarks modèles locaux sur RTX 3090

Modèles retenus pour la stack Jarvis (choix final à affiner en Phase 1bis) :

Modèle	Taille VRAM	Vitesse	Rôle envisagé
Qwen 3 32B Q4_K_M	22,2 Go	~35 tok/s	Tâches standard, qualité proche Claude Sonnet
Qwen 3 30B-A3B (MoE)	~20 Go	~130 tok/s	Ultra-rapide pour routing/résumé/classification
Qwen 2.5 Coder 32B	~20 Go	~30 tok/s	Génération YAML HA, scripts PowerShell, Python
Llama 3.1 8B Q4	~5 Go	112 tok/s	Brouillons, filtrage email, tri rapide

Modèle	Taille VRAM	Vitesse	Rôle envisagé
DeepSeek-R1 14B	~9 Go	56 tok/s	Raisonnement chain-of-thought, maths

Stockage total estimé : **~80 Go** pour l'ensemble des modèles, sur un SSD NVMe idéalement.

2.3 Claude (via Plan Max, conservé)

Aucun changement côté Claude. Le plan Max reste la **colonne vertébrale** du projet pour tout ce qui demande créativité, raisonnement complexe, sessions exploratoires longues, et interaction riche via Cowork Desktop.

Ce qui change, c'est la **répartition de l'usage** : on cesse d'appeler Claude pour des tâches où un modèle local ferait aussi bien (classification d'email, extraction de données simples, résumé de texte court). Claude redevient le **LLM premium pour les cas premium**, selon la philosophie "last resort" d'Hermes Agent.

Modèles utilisés : **Claude Sonnet 4.6** pour la majorité des sessions, **Claude Opus 4.7 [1M contexte]** pour les cas qui demandent un contexte particulièrement long (ce qui devient rare si la compression Hermès est bien configurée).

2.4 OpenRouter (optionnel, pay-as-you-go)

OpenRouter est un aggregator API donnant accès à 300+ modèles via une seule clé et une seule API. Mode **pay-as-you-go** : pas d'abonnement flat, on paie uniquement les tokens consommés, aux tarifs des providers (souvent moins 10 à 20 % grâce aux négociations OpenRouter).

Intérêt dans la stack Jarvis : **switch ponctuel** quand un modèle spécifique apporte un plus sur une tâche précise :

- **Gemini 2.5 Flash** (~0,15 \$/M input + 0,60 \$/M output) — **ultra-économe** pour du volume, contexte long
- **GPT-5 mini** (~0,25 \$/M input + 2 \$/M output) — **très économe** sur tâches simples
- **DeepSeek V3** (~0,14 \$/M input + 0,28 \$/M output) — **le moins cher**, excellent rapport qualité/prix

Dépôt initial estimé : **5-10 \$**, consommation prévisionnelle **5-10 \$/mois** (à ajuster selon usage réel).

Intégration : clé API dans ~/.hermes/config.yaml, Hermès route automatiquement selon les seuils configurés.

2.5 Obsidian (mémoire partagée, optionnel)

Obsidian est un gestionnaire de notes markdown avec graph view, backlinks, et écosystème de plugins. Dans la stack Hermès, il joue **deux rôles** :

- **Visualiseur** du vault (graph view, backlinks automatiques, recherche rapide)
- **Interface d'édition humaine** si Mickael veut intervenir manuellement sur le wiki

Obsidian n'est pas obligatoire — Hermès fonctionne avec un simple dossier markdown. Mais il apporte un confort visuel non négligeable pour voir la mémoire de Jarvis “de l'extérieur”.

Plugins pertinents :

- **Dataview** — requêtes dynamiques sur le frontmatter YAML (tableaux auto, TODOs)
 - **Web Clipper** — extension navigateur pour sauver une page web en markdown dans le vault d'un clic
 - **Marp** — génération de présentations depuis un fichier markdown
-

Chapitre 3 — Comment ça fonctionne

3.1 Le principe du routing intelligent

Le cœur du système est Hermès qui **décide où envoyer chaque requête**. Le principe est simple : **Claude en dernier recours**.

À la réception d'une requête (peu importe le canal d'entrée — PC Cowork, iPhone HA, Telegram, CLI...), Hermès analyse :

1. La **complexité** de la tâche (0-10)
2. La **valeur ajoutée potentielle** d'un appel Claude (0-10)
3. Le **type** de tâche (factuel, raisonnement, créatif, code...)
4. Le **contexte mémoire** déjà disponible dans Obsidian

Puis il route :

- Si complexité < 6 et valeur < 7 → **modèle local** (Ollama, RTX 3090)
- Si raisonnement complexe requis → **Claude Sonnet / Opus** via Plan Max
- Si cas spécifique (contexte très long, volume important) → **OpenRouter** (Gemini Flash, DeepSeek V3)

Les seuils sont configurables dans `~/hermes/config.yaml`. Au démarrage on part avec des valeurs conservatrices, on les ajuste après 1-2 semaines d'usage.

3.2 Les fallbacks dynamiques

Les seuils statiques ne suffisent pas. Le système doit aussi surveiller **des signaux runtime** :

- Si le **GPU est saturé** (autre tâche en cours) → fallback Claude
- Si la **latence locale dépasse un seuil** (par exemple > 3 s pour un premier token) → fallback Claude
- Si le **modèle local renvoie une réponse trop courte** ou détecte son incompétence → fallback Claude
- Si Mickael demande **explicitement** un modèle précis → respect de la préférence

Ces fallbacks rendent Hermès **robuste**. En pratique, 80 % des tâches restent locales, mais le système ne bloque jamais sur un cas limite — il escalade automatiquement.

3.3 Les trois niveaux de mémoire

Hermès tient une mémoire persistante à trois niveaux, avec recherche FTS5 full-text :

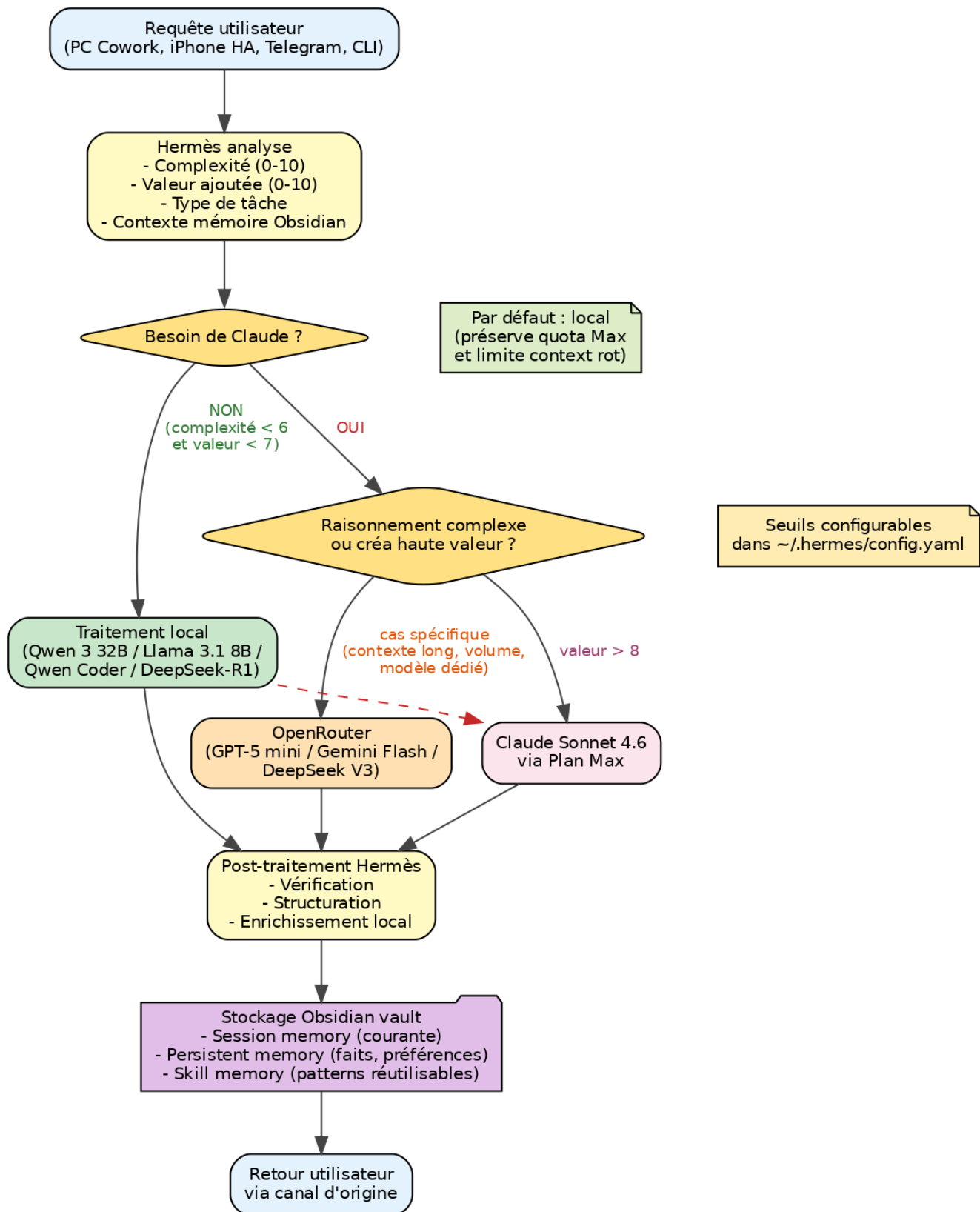


Figure 4: Workflow de routing — décision requête par requête

- **Session memory** — la conversation en cours. Effacée au redémarrage.
- **Persistent memory** — les faits, préférences, décisions qui traversent les sessions. Équivalent exact de `.auto-memory/` dans Jarvis actuel. Exemples : “Mickael préfère les réponses détaillées par défaut sauf sur iPhone”, “La boîte `might57290@gmail.com` est celle du tri principal”.
- **Skill memory** — les patterns de résolution. Équivalent de `.claude/skills/` mais **auto-générés** après qu’Hermès ait résolu une tâche complexe d’une manière réutilisable.

Les trois niveaux sont consultables par recherche sémantique, et Hermès charge dans le contexte uniquement ce qui est pertinent pour la requête en cours — ce qui évite l’explosion de tokens au démarrage.

3.4 Les skills auto-crées

C’est une des fonctionnalités les plus intéressantes de Hermes Agent. Après une tâche complexe résolue avec succès (par exemple “générer un YAML d’automation HA pour une règle spécifique”), Hermès peut :

1. **Extraire le pattern** de la solution
2. **Écrire une skill** qui documente les étapes
3. **Sauvegarder dans Skill memory**
4. **Relancer la skill automatiquement** la prochaine fois qu’une tâche similaire arrive

Ça crée un **effet compounding** : plus Jarvis tourne, plus il devient efficace sur les tâches récurrentes, sans intervention humaine pour écrire les skills.

3.5 Les gateways — pilotage depuis n’importe où

Hermès expose son interface sur de nombreux canaux, activables ou non selon besoin.

Pour la stack Jarvis spécifiquement, les gateways **prioritaires** seraient :

- **Home Assistant natif** — l’app HA sur iPhone devient **l’interface mobile principale**. Plus besoin de passer par `claude.ai` mobile.
- **Telegram** — Mickael l’a validé comme bonne option. Permet de parler à Jarvis depuis n’importe où, même sans HA ouvert.
- **CLI / Terminal** — accès direct sous WSL2 pour les tâches admin.
- **Webhook / cron** — intégration avec les scheduled tasks Windows existantes.
- **Email** — si on veut garder un canal email parallèle (moins prioritaire vu que le tri email est déjà géré via Gmail MCP).

Les autres gateways (Discord, Slack, WhatsApp, Signal, SMS, Matrix, iMessage) restent disponibles mais **ne seront pas activés** — pas d’usage identifié pour Mickael.

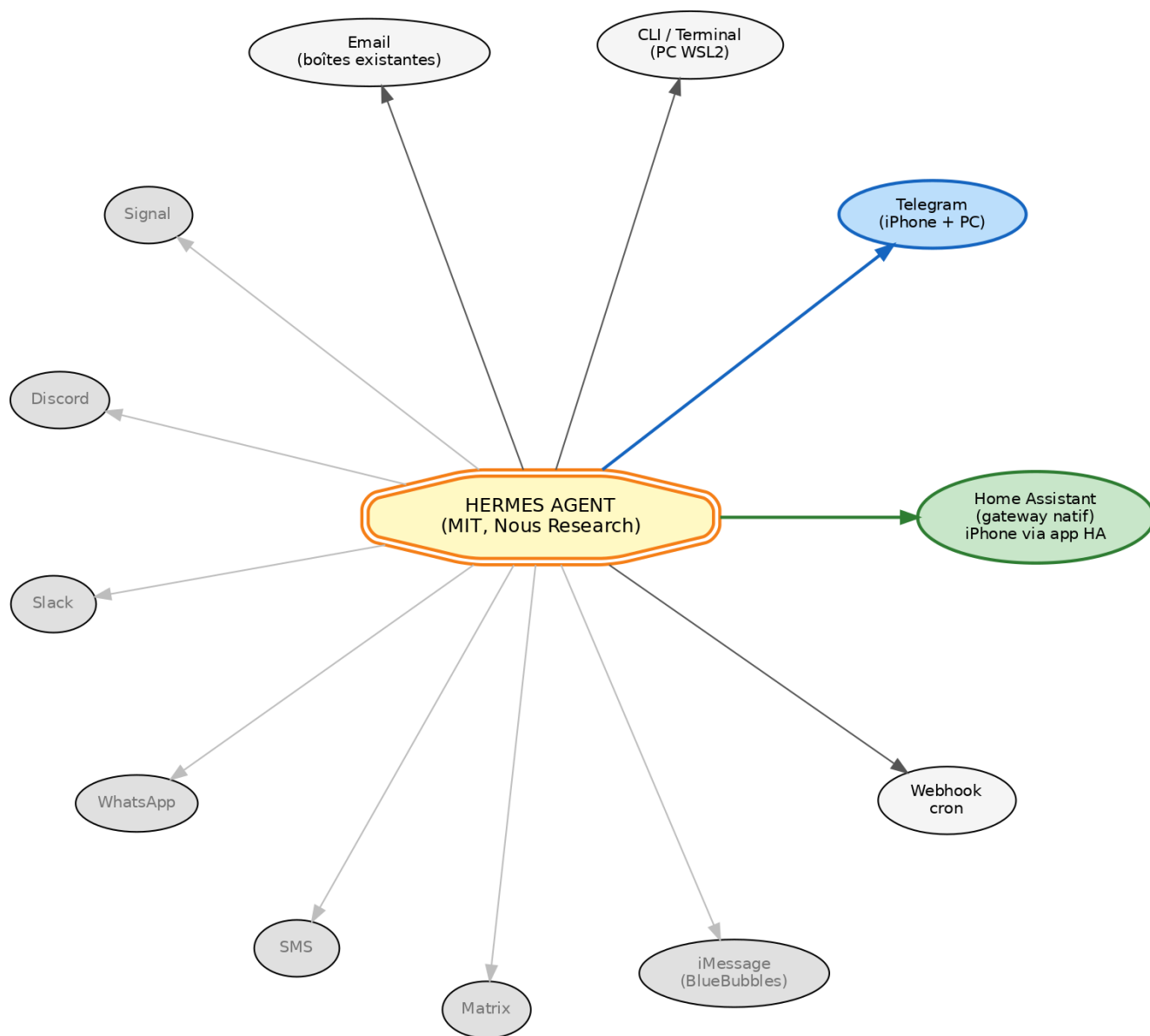


Figure 5: Gateways Hermès — points d'entrée multi-canaux

Chapitre 4 — Ce que ça change pour toi

4.1 Répartition des appels LLM — avant / après

Aujourd'hui, **100 %** des appels LLM de Jarvis consomment le quota Claude Max. Après adoption d'Hermès, la cible visée est :

- ~**75 %** des tâches traitées **localement** (Ollama sur RTX 3090, gratuit, instantané)
- ~**20 %** sur **Claude Max** (ce qui demande vraiment la qualité Claude)
- ~**5 %** sur **OpenRouter** (cas spécifiques pay-as-you-go)

Répartition prévisionnelle des appels LLM après adoption Hermès

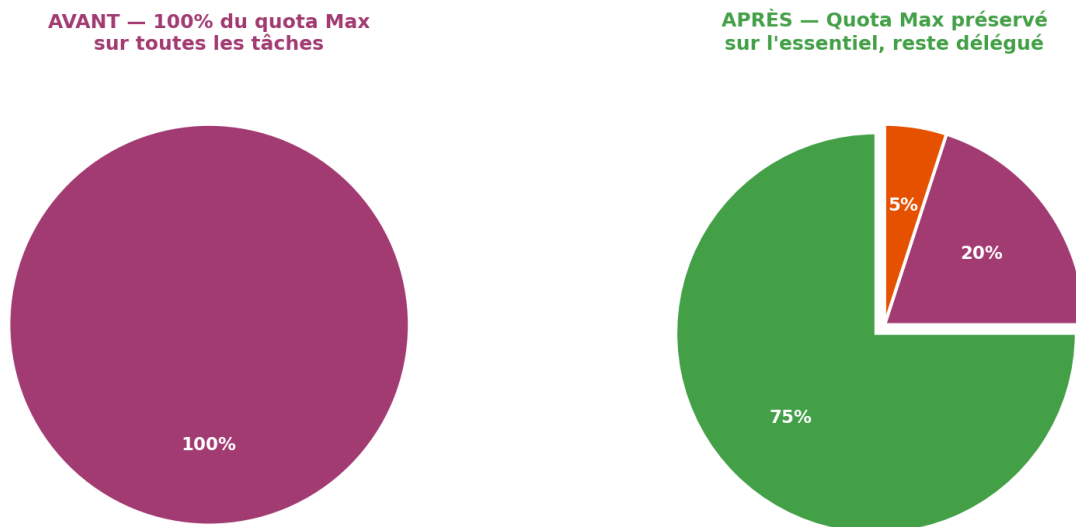


Figure 6: Répartition tokens avant / après adoption Hermès

Concrètement :

- Le tri email quotidien ? Local.
- Le filtrage des alertes Mode Réactif ? Local.
- La classification d'un email en Urgent/Perso/Info ? Local.
- La génération d'un YAML simple pour une automation HA ? Local.
- La rédaction d'un brouillon d'email professionnel avec nuances ? Claude.

- Un debug complexe d'automation HA avec multi-étapes ? Claude.
- Un brainstorming stratégique long ? Claude.
- Un résumé d'article web de 10 000 mots ? Gemini Flash (OpenRouter) — contexte long pas cher.

Le quota Max redevient **abondant pour les cas qui comptent**, au lieu d'être grignoté par les tâches routinières.

4.2 Cohabitation Cowork ↔ Hermès (pas de remplacement)

Un point important : **Hermès ne remplace pas Cowork**. Les deux cohabitent, chacun sur son terrain.

Cowork Desktop garde sa valeur unique sur : - Les sessions créatives exploratoires riches - Les workflows visuels (artefacts, visualisations, HTML interactif) - Les MCPs avancés Cowork-only (Claude in Chrome, PDF Tools interactifs) - Les skills créatives complexes auto-déclenchées - L'édition directe des fichiers projet via Read/Edit/Write/Bash

Hermès prend le relais pour : - Le pilotage mobile 24 h/24 (HA + Telegram) - Les tâches récurrentes (Mode Réactif, tri, rapports) - La mémoire persistante partagée - L'orchestration multi-LLM

Les deux outils partagent le même **vault Obsidian** — ce qui veut dire qu'une décision prise dans Cowork (par exemple une nouvelle règle de tri) est immédiatement connue d'Hermès, et réciproquement.

4.3 Pilotage mobile vraiment natif

Aujourd'hui, quand Mickael est en déplacement, il ouvre **claude.ai mobile** — qui est bien mais reste une interface de chat générique. Avec Hermès :

- **App Home Assistant sur iPhone** — Mickael parle à Jarvis dans la même app qui contrôle sa maison. Cohérence totale.
- **Telegram** — pour parler à Jarvis sans même ouvrir HA (pratique pour une question rapide).

Et en bonus : le PC Mickael étant allumé 24 h/24, Hermès tourne en **daemon permanent**. Plus besoin d'avoir Cowork ouvert en continu.

4.4 Préservation du quota Max

Le forfait Claude Max est un **actif** à préserver. Il a des limites (3000 messages/semaine pour certains modèles selon le plan, quota tokens implicite). Aujourd'hui ce quota est consommé par des tâches que Claude sur-qualifie (lire un email court, classer, compter).

Avec Hermès, le quota Max est **libéré pour les moments qui le méritent** : les sessions de brainstorming, les audits de sécurité profonds, les refontes d'architecture, la rédaction créative.

L'objectif n'est pas de "payer moins" — Max est déjà payé. L'objectif est d'**utiliser mieux** ce qu'on paie, et de ne pas se retrouver bloqué par un quota consommé sur des tâches insignifiantes.

Chapitre 5 — Coûts et retour sur investissement

5.1 Comparatif des stratégies d'abonnement

Plusieurs combinaisons ont été évaluées :

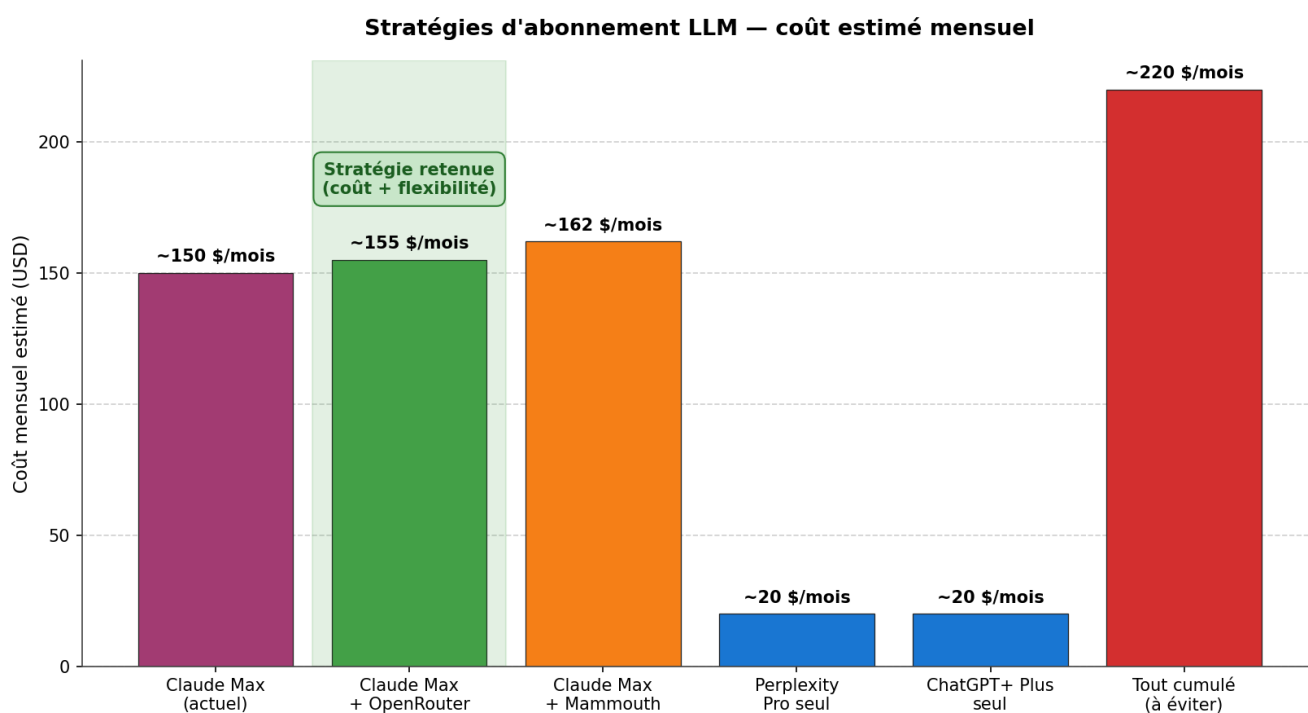


Figure 7: Stratégies d'abonnement LLM — coût mensuel estimé

La stratégie retenue : Claude Max (conservé) + OpenRouter (pay-as-you-go à la consommation réelle). Budget estimé :

- Claude Max : **~150 \$/mois** (déjà souscrit)
- OpenRouter : **~5-10 \$/mois** de consommation prévisionnelle
- Ollama + modèles locaux : **0 \$** (électricité négligeable, PC déjà allumé)
- Hermes Agent : **0 \$** (open source MIT)
- Obsidian : **0 \$** (usage personnel gratuit)

Total additionnel estimé par rapport à l'existant : ~5-10 \$/mois.

5.2 Ce qui a été écarté

- **Mammoth AI** (~10-12 €/mois flat, UI chat français) — écarté car pas d'API publique, donc inutilisable dans Hermès. Redondant avec claude.ai mobile.
- **Perplexity Pro** (20 \$/mois, UI multi-LLM + deep research) — écarté car redondant avec Claude Max + Hermès/Telegram prévu.
- **ChatGPT Plus** (20 \$/mois, GPT-5.4) — écarté, redondance totale.
- **Gemini Advanced** (20 \$/mois, Gemini 3 Pro) — écarté, Gemini Flash via OpenRouter fait l'essentiel pour moins cher.
- **Aucun 2ème abonnement flat** ne sera souscrit sans raison spécifique identifiée.

5.3 Budget temps d'installation

- **Phase 1bis** (étude détaillée) : 1 session dédiée, ~1 h
- **Phase 2** (installation effective) : 2-3 sessions, ~3-5 h au total
 - Install WSL2 + Ollama + Hermès
 - Téléchargement modèles (~80 Go)
 - Configuration config.yaml + MCPs + gateways
 - Tests V1 sur 2-3 tâches pilotes
- **Phase 3** (migration progressive) : au fil de l'eau, semaines à mois
 - Chaque skill Jarvis est migrée une à une sous Hermès, avec validation sur 1 semaine avant généralisation
 - Aucun "big bang" — Cowork reste opérationnel tout au long

5.4 Budget temps récurrent

Une fois en production, le coût de maintenance est faible :

- **Skill auto-crée**e par Hermès — aucune intervention humaine
 - **Mise à jour Hermès** (MIT, releases tous les 2-4 semaines) — 10 min par mise à jour
 - **Mise à jour modèles Ollama** (nouveaux releases, optionnelle) — 15 min par modèle
 - **Ajustement seuils config.yaml** — 30 min tous les 1-2 mois selon feedback
-

Chapitre 6 — Phasage et timeline

6.1 Vue d'ensemble des phases

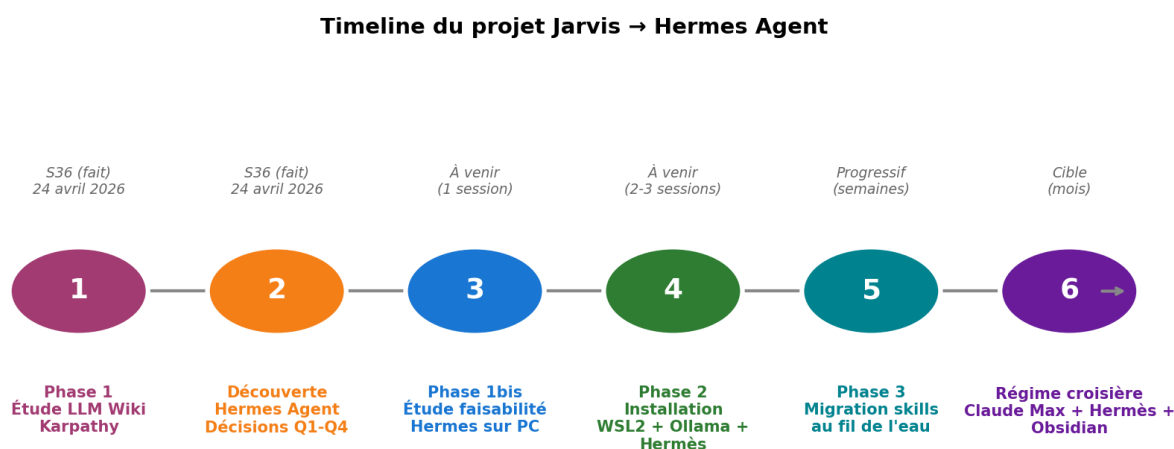


Figure 8: Timeline du projet Jarvis → Hermes Agent

6.2 Détail de chaque phase

Phase 1 — Étude LLM Wiki Karpthy (fait, session 36)

Livrable : rapport de 12 pages Projets/Jarvis_Hermes_Projet/Rapport_Phase1.md + PDF équivalent.

Conclusion : refonte complète en pattern Karpthy pur = NON (Jarvis déjà à 130-165 .md dans la zone où Zafer Dace a documenté une panne à 80-100 articles ; doublon à 90 % avec l'infrastructure Jarvis existante ; mismatch entre le cas d'usage encyclopédique du pattern et le cas d'usage opérationnel de Jarvis).

Découverte Hermes Agent (fait, session 36)

Suite aux questions de Mickael (Q1-Q7), recherche intensive sur "Hermès + Claude + Obsidian".
Découverte majeure : Hermes Agent existe déjà, produit Nous Research MIT lancé 02/2026, qui

implémente nativement ce que le schéma de Mickael décrivait. On ne fabrique rien, on installe un produit existant.

Livrible : 6 auto-memories créées, archive session exhaustive, décisions Q1-Q4 validées par Mickael.

Phase 1bis — Étude de faisabilité Hermes sur la config Mickael (à venir)

Durée estimée : 1 session dédiée (~1 h).

Pré-requis : Mickael partage un descriptif PC complet (Speccy Publish Snapshot, DxDiag complet, ou HWiNFO64). Le DxDiag partiel reçu en session 36 confirme l'essentiel (i9-9900K, 32 Go RAM, Windows 11 Pro build 26200, dGPU présent) mais la section Display Devices manque pour la VRAM RTX 3090 exacte, et Disk pour le stockage NVMe.

Livrible attendu : rapport Projets/Jarvis_Hermes_Projet/Rapport_Phase1bis.md détaillant le plan d'installation pas-à-pas, la liste exacte des modèles à télécharger, le plan de migration des skills Jarvis, le budget tokens prévisionnel, et l'application de la Règle 0.

Phase 2 — Installation effective (à venir)

Durée estimée : 2-3 sessions, ~3-5 h cumulées.

Étapes :

1. Installation WSL2 Ubuntu 24.04 sur le PC Windows (si pas déjà fait)
2. Installation NVIDIA CUDA drivers pour WSL2 (GPU passthrough)
3. Installation Ollama dans WSL2, configuration
4. Téléchargement modèles (ollama pull qwen3:32b, etc.)
5. Installation Hermes Agent (curl install.sh | bash)
6. Configuration ~/.hermes/config.yaml — providers, MCPs, gateways
7. Activation gateway Home Assistant (add-on HA à installer ou webhook)
8. Activation gateway Telegram (création bot, token, config)
9. Test V1 sur 1 tâche simple (tri d'un email de test)
10. Validation Mickael → Phase 3

Phase 3 — Migration progressive des skills (semaines à mois)

Chaque skill Jarvis actuelle (.claude/skills/*) est migrée **une à une** sous Hermès :

- Priorité 1 : tri-email-gmail (ROI le plus fort, tourne plusieurs fois par jour)
- Priorité 2 : check-jarvis-alert (Mode Réactif, tourne quotidiennement)
- Priorité 3 : rapport-journalier-reactif (tourne quotidiennement)
- Priorité 4+ : les autres skills, selon fréquence d'usage

Pour chaque migration :

1. Skill dupliquée sous Hermès (Skill memory)
2. Test en parallèle sur 1 semaine (Cowork + Hermès tournent tous les deux, comparaison des résultats)
3. Si Hermès donne résultats équivalents ou meilleurs → la skill Cowork est marquée OBSOLETE
4. Si problème détecté → rollback immédiat, investigation

Principe : Cowork reste **source de vérité** tant que la skill Hermès n'est pas validée. Aucun risque de perte fonctionnelle.

Régime croisière (cible)

Une fois la migration stabilisée :

- Hermès tourne 24 h/24 en daemon WSL2
- Cowork s'ouvre à la demande pour les sessions créatives riches
- Mickael pilote Jarvis principalement via l'app HA sur iPhone (mobile) ou via Cowork (PC)
- Claude Max est préservé, OpenRouter consomme peu
- La mémoire d'Hermès compound au fil des semaines

6.3 Points de décision côté Mickael

Quatre décisions formelles sont attendues :

1. **Lancer Phase 1bis ou reporter** → décision à prendre ponctuellement
2. **Lancer Phase 2** → après lecture rapport Phase 1bis
3. **Budget OpenRouter** → création compte + dépôt 5-10 \$ initial
4. **Priorité de migration** → valider l'ordre de migration des skills en Phase 3

Aucune de ces décisions n'est urgente ni irréversible.

Chapitre 7 — Risques et garde-fous

7.1 La Règle 0 s'applique partout

La règle prioritaire de Jarvis reste en vigueur dans l'architecture Hermès :

Avant tout accès à un mot de passe, token, clé API, ou donnée sensible, Jarvis s'arrête, décrit ce qui serait vu, propose à Mickael de le faire lui-même, ou demande explicitement l'accès.

Concrètement pour Hermès :

- **Secrets OAuth** (Gmail, Google Calendar, Cloudflare...) restent sous `~/.gmail-mcp/`, `~/.ha-config/` avec ACL NTFS/chmod strictes. Hermès les lit via les MCPs qui les encapsulent, jamais en direct.
- **Credentials OpenRouter** (sk-or-xxxxx) stockés dans `~/.hermes/secrets/` avec chmod 600, jamais loggés, jamais envoyés à un LLM distant.
- **Denylist Hermès** — certains outils MCP critiques (suppression, envoi email non-notify, delete file) sont en denylist stricte, même pour Hermès.

7.2 Failure modes et mitigation

Risque 1 — Hermès se plante (crash, bug, release cassée) *Mitigation* : Cowork reste opérationnel comme fallback immédiat. Aucune dépendance de Cowork vers Hermès. En cas de crash Hermès, Mickael continue via Cowork sans interruption.

Risque 2 — Un modèle local donne une réponse fausse avec confiance *Mitigation* : fallback dynamique Hermès vers Claude si le modèle local détecte son incompetence ou si la réponse est trop courte. Règle : pour toute action d'écriture HA ou envoi email, **Claude (plus fiable) est privilégié**. Le local est utilisé pour la lecture, le tri, la classification, mais pas pour les actions critiques sans validation.

Risque 3 — Le GPU est saturé par une autre tâche (jeu, rendu 3D, autre calcul) *Mitigation* : Hermès détecte la charge GPU (monitoring nvidia-smi) et fallback vers Claude. Aucun blocage.

Risque 4 — Dérive de la mémoire persistante (faux souvenirs, contradictions) *Mitigation* : Hermès embarque nativement un Lint périodique (hérité du pattern Karpathy) qui audite la mémoire. Planifié toutes les 2-4 semaines. Les contradictions sont flaguées pour revue humaine avant correction.

Risque 5 — Skill auto-crée qui fait n'importe quoi *Mitigation* : nouvelle skill auto-crée = statut "probation" par défaut. Elle tourne pendant 1 semaine en mode shadow (l'action Claude reste source de vérité, la skill est comparée) avant d'être promue en production.

Risque 6 — Consommation OpenRouter qui explose *Mitigation* : budget mensuel plafonné (hard limit côté OpenRouter), alerte HA si consommation journalière > seuil, coupure automatique si dépassement majeur.

7.3 Plan de rollback

À tout moment, Mickael peut décider de revenir à l'architecture Cowork-only :

1. Désactiver les gateways Hermès (HA, Telegram) — 2 min
2. Arrêter le service Hermès sous WSL2 (`systemctl stop hermes` ou `hermes stop`) — 1 min
3. Réactiver les scheduled tasks Cowork pour les tâches récurrentes — 5 min
4. Les fichiers Cowork (`.claude/skills`, `.auto-memory`, `CLAUDE.md`) n'ont jamais été touchés par Hermès (principe de non-régression) — donc rien à restaurer

Point d'attention : si la mémoire persistante d'Hermès a évolué pendant la période Hermès (faits appris, préférences raffinées), ces informations ne sont pas automatiquement dans `.auto-memory/`. Il faut prévoir une **étape d'extraction** pour rapatrier les apprentissages d'Hermès vers Jarvis Cowork avant de désactiver Hermès complètement.

Conclusion

Synthèse

Le projet Hermes Agent pour Jarvis :

- **Résout un vrai problème** — consommation quota Max sur des tâches qui ne le méritent pas, dépendance cloud totale, mobilité via claude.ai fallback générique.
- **S'appuie sur du concret** — Hermes Agent existe (MIT), Ollama existe, RTX 3090 existe (chez Mickael), communauté existe (10+ setups publics documentés).
- **Préserve l'existant** — Cowork Desktop garde sa valeur, pas de refonte destructrice, migration progressive skill par skill.
- **A un coût maîtrisé** — +5 à 10 \$/mois en OpenRouter, 0 \$ ailleurs, 3-5 h d'installation.
- **Expose des garde-fous solides** — Règle 0 appliquée, rollback à tout moment, fallbacks multiples.

Prochaines actions

Action Mickael n°1 — *optionnelle, à ton rythme* — partager un descriptif PC complet pour Phase 1bis. Méthode recommandée : Speccy + Publish Snapshot (lien URL partageable en 2 min), ou DxDiag complet avec toutes les sections.

Action Mickael n°2 — *optionnelle, quand tu veux* — décider si on lance Phase 1bis cette semaine, le mois prochain, ou plus tard. Aucune urgence.

Action Jarvis — *automatique dès prochaine session* — le projet est documenté dans `project_hermes_agent_phaselbis.md` (auto-memory) et `TASKS.md` #58. Jarvis le relancera naturellement quand Mickael le voudra.

Dernière précision

Ce document est **une vision**, pas une obligation. Rien ne dit qu'Hermes Agent est la seule voie ni la meilleure pour toi. Il est possible que :

- Après Phase 1bis, tu trouves que la complexité d'installation n'en vaut pas la peine.
- Après 1 mois d'usage Phase 2, tu trouves que le gain est modeste par rapport au setup.
- Tu décides de ne faire que l'**Obsidian visualiseur pur** (Priorité 1 du rapport Phase 1) sans Hermès, pour bénéficier du graph view sans toucher à l'architecture.

Toutes ces trajectoires sont ouvertes. Ce document propose **une direction cohérente**, basée sur les technologies mûres à date (avril 2026) et sur tes contraintes personnelles (matériel disponible, Plan Max souscrit, désir de mobilité native).

La décision finale reste la tienne, en tout temps.

Annexes

A — Glossaire technique

Terme	Définition
Agent	Programme qui utilise un LLM pour accomplir des tâches de manière autonome (pas juste répondre à des questions).
AGENTS.md	Fichier schema de Hermes Agent, équivalent du CLAUDE.md de Jarvis — identité, règles, workflows.
FTS5	Full-Text Search version 5, moteur de recherche textuelle dans SQLite. Très rapide sur les gros volumes de markdown.
Harness Engineering	Philosophie Nous Research : le progrès 2026 vient du “harnais” autour du LLM, pas du LLM seul.
LLM Wiki	Pattern Karpathy : wiki markdown maintenu automatiquement par un LLM via Ingest / Query / Lint.
MCP	Model Context Protocol (Anthropic, 2024). Standard de connexion entre LLM et outils externes.
MoE (Mixture of Experts)	Architecture LLM où seule une fraction des paramètres est activée par requête, permettant des vitesses élevées à grande taille.
Ollama	Runtime local standard pour l’inférence LLM, open source.
OpenRouter	Aggregator API donnant accès à 300+ modèles via une seule clé et un seul endpoint HTTP.
Plan Max	Abonnement Anthropic grand utilisateur, inclut Claude Code CLI headless sans facturation API séparée.

Terme	Définition
Q4_K_M	Quantification 4-bit K-quants, format standard pour compresser un LLM et le faire tenir en VRAM réduite sans perte majeure de qualité.
RAG	Retrieval-Augmented Generation — pattern classique : retrieval par embeddings + génération. Ce que Karpathy veut remplacer.
Smart Model Routing	Fonctionnalité Hermes v0.10+ qui choisit automatiquement le LLM selon complexité/valeur.
Tok/s	Tokens par seconde, unité de mesure de la vitesse de génération d'un LLM. ~30 tok/s = conversationnel, > 100 tok/s = instantané.
VRAM	Mémoire dédiée du GPU. La RTX 3090 en a 24 Go, ce qui détermine quelle taille de modèle peut tourner.
WSL2	Windows Subsystem for Linux 2. Permet de faire tourner Ubuntu natif sur Windows avec GPU passthrough CUDA.

B — Configuration PC Mickael

Composant	Valeur
Hostname	MIGHT-1000D
Carte mère	ASUS (Z390-like, BIOS 1802 UEFI)
CPU	Intel Core i9-9900K @ 3,6 GHz (8 cores / 16 threads, Coffee Lake 2018)
RAM	32 Go
GPU	NVIDIA RTX 3090 24 Go VRAM (confirmée oralement, détails Phase 1bis)
OS	Windows 11 Professionnel 64-bit, build 26200 (24H2 / 25H1)
DirectX	12
Disponibilité	24 h/24, 7 j/7
Dossier projet	D:\Might\IA\Projets Cowork\Jarvis - Home Assistant

C — Sources et références

Hermes Agent :

- Site officiel : hermes-agent.nousresearch.com

- Documentation complète : hermes-agent.nousresearch.com/docs
- Repo GitHub MIT : github.com/nousresearch/hermes-agent
- Quickstart : hermes-agent.nousresearch.com/docs/getting-started/quickstart
- AI Providers (multi-LLM) : hermes-agent.nousresearch.com/docs/integrations/providers
- Configuration yaml : hermes-agent.nousresearch.com/docs/user-guide/configuration
- Integration Ollama : docs.ollama.com/integrations/hermes

Stack complète Mickael-compatible :

- Repo communautaire Hermes + Claude Cowork + Obsidian : github.com/omankz/Hermes-Agent—Claude-Cowork—Notion—Obsidian
- Playbook Claude Cowork + Obsidian : github.com/aodhanzm/AeroLex-AI-Playbooks
- xda-developers — Claude Code dans Obsidian : xda-developers.com/claude-code-inside-obsidian

Karpathy LLM Wiki (inspiration backend Hermès) :

- Gist canonique : gist.github.com/karpathy/442a6bf555914893e9891c11519de94f
- Retour terrain Aaron Fulkerson : aaronfulkerson.com — LLM Wiki in Production
- Retour terrain Zafer Dace (panne à 100 articles) : earezki.com — Karpathy's Obsidian wiki broke at 100 articles

Benchmarks RTX 3090 :

- Guide battle-tested Ubuntu + RTX 3090 : github.com/keturk/llm_on_rtx_3090
- Benchmarks 32B modèles 24 Go : modelfit.io/gpu/rtx-3090
- Deployment local 24 Go : intuitionlabs.ai/articles/local-llm-deployment-24gb-gpu-optimization

Abonnements LLM :

- OpenRouter pricing : openrouter.ai/pricing
- AI pricing 2026 comparé : findskill.ai — AI pricing comparison 2026
- ChatGPT Plus pricing 2026 : chatgptpluspricing.com

Documents Jarvis internes référencés :

- Projets/Jarvis_Hermes_Projet/Rapport_Phase1.md — rapport étude Phase 1 LLM Wiki (503 lignes, 40 Ko)
- Projets/Jarvis_Hermes_Projet/Rapport_Phase1.pdf — équivalent PDF 12 pages
- memory/historique/2026-04-24_session_36_etude_llm_wiki_hermes_agent.md — archive exhaustive session 36
- .auto-memory/project_hermes_agent_phaselbis.md — initiative projet
- .auto-memory/reference_hermes_agent.md — fiche technique
- .auto-memory/feedback_local_llm_faisable_rtx3090.md — règle faisabilité locale
- .auto-memory/reference_pc_config_might.md — config PC
- .auto-memory/reference_llm_subscriptions_comparison.md — comparatif abonnements
- .auto-memory/feedback_mammoth_vs_openrouter.md — règle Mammoth vs OpenRouter
- TASKS.md #58 — tâche de suivi Phase 1bis

Fin du document — Projet Jarvis → Hermes Agent — Session 36, 24 avril 2026.

Rien dans ce document n'engage une décision. Tout reste ouvert.